

Trabajo Integrador – Programación 2

Sistema de Gestión de Seguros Vehiculares

Carrera: Tecnicatura Universitaria en Programación

Universidad: UTN

Fecha: 17-11-2025

Juan José Silva comisión 17

Fabián Alejandro Salas comisión 17

Luca Arlettaz comisión 2

Integrante 4 (Abandono a último momento)

Integrantes y roles

El trabajo fue realizado por los **tres integrantes**, trabajando de manera colaborativa y simultánea a través de sesiones grupales.

No hubo una división rígida de roles: **todos participamos en cada etapa del desarrollo**, incluyendo:

creación de la base de datos

conexión JDBC

diseño de entidades, etc...

Forma de trabajo

El proyecto lo realizamos en modalidad **colectivamente**, usando sesiones grupales para:

- escribir código juntos
- revisar consultas SQL
- probar transacciones
- solucionar errores entre todos
- completar el UML y el informe

Esto nos permitió que todos los integrantes aprendieran cada parte del proceso y que el trabajo fuera uniforme.

El dominio elegido fue:

Gestión de Vehículos y Seguro Vehicular (relación 1→1)

La elección se basó en que:

- Fue un dominio sencillo ya que es algo cotidiano
- permite CRUD completos
- se puede probar fácilmente con datos y desde SQL
- es suficientemente complejo para aplicar buenas prácticas sin volverse demasiado grande

El dominio replica un caso real de compañías de seguros:
cada vehículo puede tener **un solo seguro activo**.

Esto nos permitió aplicar reglas de negocio auténticas.

Diseño – decisiones clave

Modelo relacional

Las tablas principales fueron:

- **Vehiculo** (PK: ID_Vehiculo)
- **SeguroVehicular** (PK: ID_Seguro – FK única hacia ID_Vehiculo)

Relación 1→1

Se decidió implementar el 1→1 usando:

✓ **FK única en la tabla SeguroVehicular**, apuntando a Vehiculo.

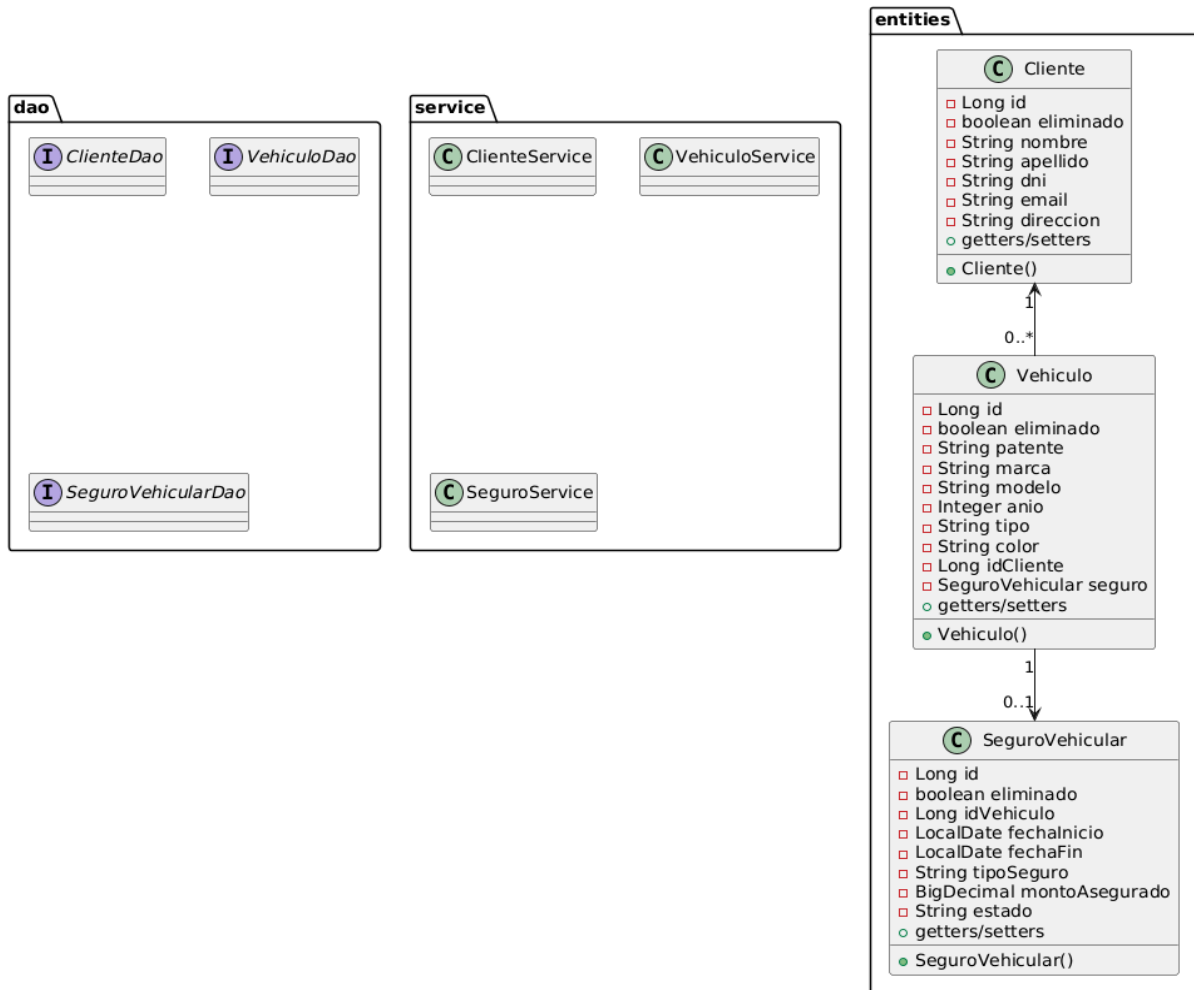
Razón:

permite que un vehículo exista sin seguro, lo cual es real, y que el seguro dependa del vehículo.

UML

El UML incluye:

- entidades
- atributos
- tipos
- relación 1→1
- métodos principales



Arquitectura por capas

El proyecto se desarrolló usando una arquitectura por capas:

Entidades (entities/)

Clases simples que representan las tablas:

- Vehiculo
- SeguroVehicular
- Cliente

Incluyen:

- atributos
- constructores
- getters/setters
- métodos equals/hashCode

DAO (dao/ y dao.impl/)

Los DAO permiten:

- CRUD básico
- lectura con join
- búsqueda por patente
- lectura de vehículo con su seguro
- lectura de seguro por id de vehículo

Incluye una clase base:

AbstractDao → entrega conexiones JDBC.

Service (service/)

La capa donde se aplican las **reglas de negocio**:

- validaciones
- transacciones
- commit / rollback
- coordinación entre VehiculoDao y SeguroDao
- método especial: *crear vehículo + seguro*

Presentación (main/AppMenu)

Interfaz de consola para el usuario:

- crear vehículo
- listar
- buscar
- eliminar
- llamar métodos del Service

Persistencia y transacciones

Base de datos

Se utilizó MySQL.

El proyecto incluye:

- **archivo SQL con creación de tablas**
- **archivo SQL con inserts de prueba**

Orden de operaciones

Para crear un vehículo:

1. Insert en Vehiculo
2. Insert en SeguroVehicular
3. Commit

Para eliminar:

- UPDATE marcando **eliminado = TRUE**

Transacción “crear vehículo + seguro”

```
conn.setAutoCommit(false);  
vehiculoDao.crear(v, conn);
```

```
boolean existe = seguroDao.existsByVehiculold(id, conn);  
if (existe) { conn.rollback(); ... }
```

```
seguroDao.crear(s, conn);  
conn.commit();
```

Situaciones que activan rollback

- si el vehículo ya tiene seguro
- si falla el insert del seguro
- si surge cualquier excepción de SQL

El rollback garantiza: Si algo sale mal - **no queda un vehículo sin seguro ni un seguro huérfano.**

Validaciones y reglas de negocio

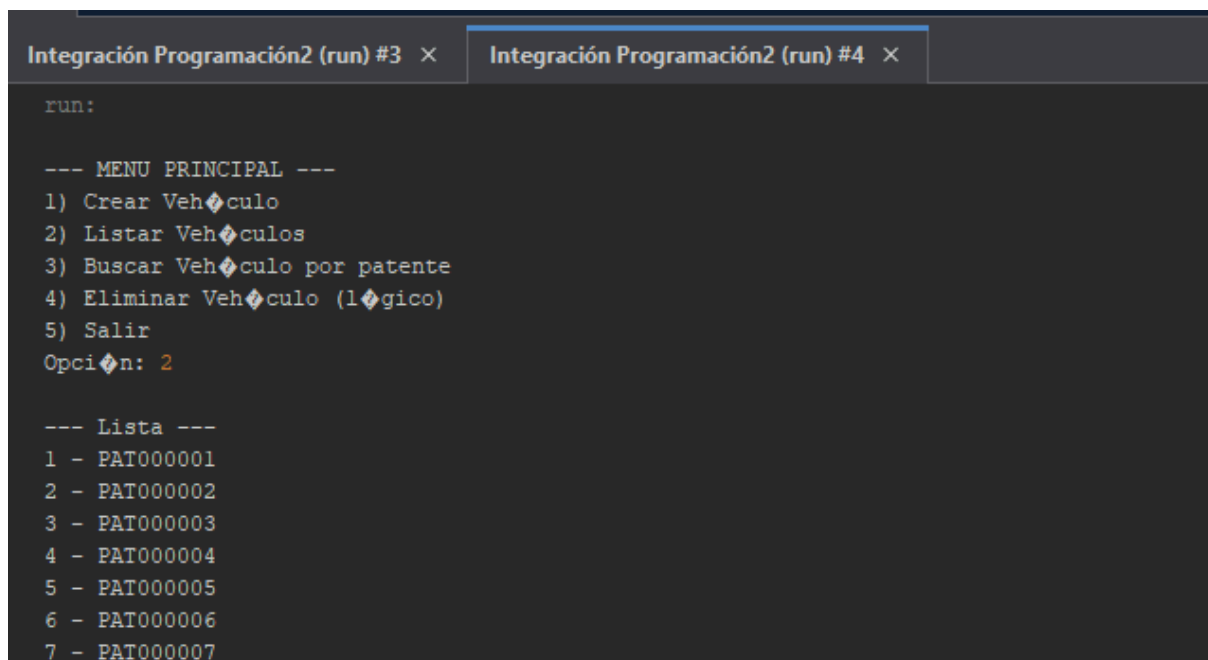
Las reglas implementadas incluyen:

- patente obligatoria y en mayúsculas
- marca/modelo no vacíos
- tipo debe ser: auto, moto o camión
- año numérico
- ID_Cliente debe existir (si hay tabla cliente)
- no se permite crear más de un seguro por vehículo
- eliminación lógica (no se borran datos del sistema)

Estas validaciones están distribuidas entre:

- AppMenu (validaciones básicas)
- Service (reglas de negocio)
- restricciones SQL (UNIQUE, NOT NULL)

Pruebas realizadas



```
run:

--- MENU PRINCIPAL ---
1) Crear Vehículo
2) Listar Vehículos
3) Buscar Vehículo por patente
4) Eliminar Vehículo (lógico)
5) Salir
Opción: 2

--- Lista ---
1 - PAT000001
2 - PAT000002
3 - PAT000003
4 - PAT000004
5 - PAT000005
6 - PAT000006
7 - PAT000007
```

```
--- MENU PRINCIPAL ---
1) Crear Vehículo
2) Listar Vehículos
3) Buscar Vehículo por patente
4) Eliminar Vehículo (lógico)
5) Salir
Opción: 3
Patente: PAT000140
Vehículo{id=140, patente='PAT000140', marca='Toyota', modelo='Sedan'}
```

```
114 • SELECT v.Patente, s.Tipo_Seguro, s.Monto_Asegurado
115 FROM Vehiculo v
116 LEFT JOIN SeguroVehicular s ON v.ID_Vehiculo = s.ID_Vehiculo;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Patente	Tipo_Seguro	Monto_Asegurado	
PAT000001	total	588297.00	
PAT000002	contra robo	578135.00	
PAT000003	responsabilidad civil	729949.00	
PAT000004	terceros	425898.00	
PAT000005	contra robo	786355.00	

Conclusiones y mejoras futuras

El grupo logró diseñar un modelo relacional correcto, implementar un vínculo 1→1 real mediante una clave foránea única, aplicar una arquitectura por capas, desarrollar CRUD completos, utilizar transacciones con commit y rollback, realizar validaciones efectivas e integrar todo en un menú funcional. Además, todos los integrantes fortalecieron buenas prácticas de DAO, el manejo de JDBC, la comprensión de la estructura por capas y la dinámica de trabajo colaborativo real.

Mejoras futuras Como próximos pasos, se propone incorporar una interfaz gráfica, implementar un sistema de login con roles de usuario, sumar el CRUD completo para la tabla Clientes, permitir múltiples seguros históricos, generar reportes en PDF y añadir un sistema de logs con manejo de errores de nivel profesional.

Fuentes y herramientas utilizadas

- Java 17 / 21
- MySQL / MariaDB
- JDBC
- GitHub
- NetBeans / IntelliJ
- IA (ChatGPT) para asistencia en explicaciones como si fuera un profesor e informe.
- Unidad 10 acceso a base de datos por el profesor e Ing. Carlos Alejandro Martinez.